The BrokerService: This is an infrastructure service that offers a way for publishers and subscribers to stay in touch with each other, asynchronously. Publishers publish events to the broker and subscribers listen for the events. A full-scale broker design is not covered at this point. Just its API classes are good enough for the publishers and subscribers like *AddService*. Normally, these kinds of infrastructure services are offered by third-party tools like Kafka. In our case, we are building these in-house.

Our *BrokerService* offers an API that consists of *Event* and *Publisher*, as presented in Figure 5.

```
package com.glarimy.broker;

public class Event {
private String topic;
private String message;

public Event(String topic, String message) {
     //TODO: validation
this.topic = topic;
this.message = message;
}

public String getTopic() {
return topic;
}

public String getMessage() {
return message;
}

}
```

And the actual implementation of the publisher is not that important for us, at this point!

```
package com.glarimy.broker;

public class Publisher {
     public void publish(Event event) {
       //TODO
     }
}
```

Generic framework: The *AddService* and many other microservices require some sort of framework to handle the common technical work. For instance, the framework may offer factories, builders, exception stations, proxies, decorators, and what not? Again, such frameworks are available in the market. In our case, we are building the framework as well. The simple model of our framework is

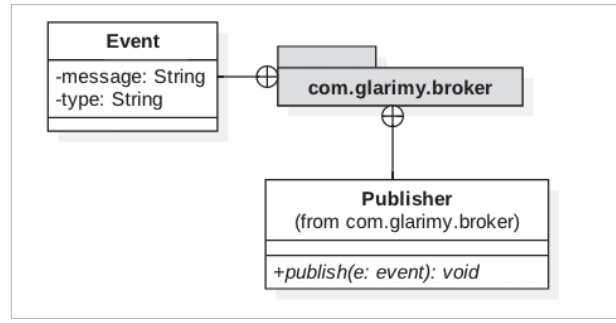

Figure 5: The API of *BrokerService*

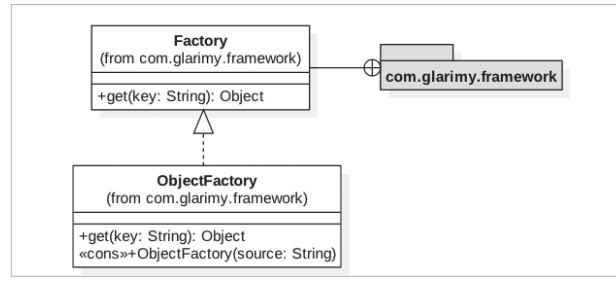

Figure 6: The framework classes

presented in Figure 6.

Since the framework is generic, it belongs to the infrastructure and many microservices (not just *AddService*) may want to use it.

Thus we designed *AddService* as a microservice, and *BrokerService* and *Framework* as reusable infrastructure services.

FindService

Once you design a microservice, you can design any number of microservices. Just like in the case of *AddService*, the *FindService* also designs relevant domain models, application layers and uses some technical services that form the infrastructure layer.

Observe Figure 7. You will notice that the entities and value objects of the domain model are more or less similar to that of the *AddService*. This happens in any microservice. Yet, the teams may not even know about it as they work fairly independently on their own platforms. For example, while *AddService* is implemented on Java, *FindService* may be implemented on Python.

Figure 8 depicts the application layer of *FindService*. This also follows the same pattern as we have seen in the case of *AddService*.

Since we have seen the code for *AddService*, there is no point in illustrating the code for the remaining services. It saves both time and space.

SearchService

By now it must be clear that designing these kinds of